

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No: 92301733022

Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression

IDE: Google Colab

Theory:

Linear regression is a method for determining the best linear relationship between two variables X and Y . If variables X and Y are uncorrelated, it is pointless embarking upon linear regression. However, if a reasonable degree of correlation exists between X and Y then linear regression may be a useful means to describe the relationship between the two variables. The usual approach is to use the *least-squares* method, which minimizes the squared difference between the actual data points and a straight line. Let $[x_i, y_i], i = 1, 2, 3, \dots, N$ be the N pairs of data values of the variables X and Y . The straight-line relating X and Y is $y = mx + c$, where m and c are the gradient and constant values (to be determined) defining the straight line. Thus, $y(x_i) - y_i$ is the difference between the line and data point i (see Fig. 1). Taking all the data points, we seek values of m and c that minimize the squared difference SD .

$$\sum_1^N [y(x_i) - y_i]^2$$

This is achieved by calculating the partial derivatives of SD with respect to m and c and finding the pair $[m, c]$ such that SD is at a minimum.

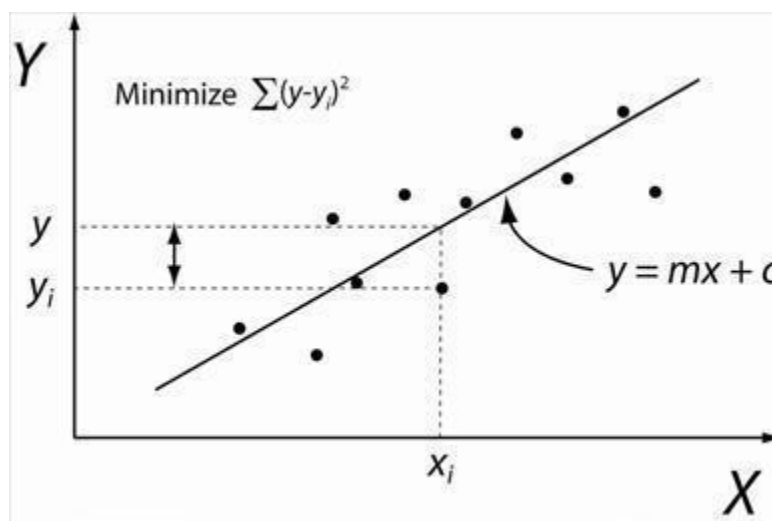


Figure 1: Illustration of Linear Regression. Linear least squares regression, the idea is to find the line $y = mx + c$ that minimizes the mean squared difference between the line and the data points

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No: 92301733022

Batch Gradient Descent:

Gradient Descent is an optimization algorithm used for minimizing the cost function in various machine learning algorithms. It is basically used for updating the parameters of the learning model. Batch gradient descent which processes all the training examples for each iteration of gradient descent. But if the number of training examples is large, then batch gradient descent is computationally very expensive.

Let m be the number of training examples. Let n be the number of features.

Algorithm for batch gradient descent :

Let $h_{\theta}(x)$ be the hypothesis for linear regression. Then, the cost function is given by: Let Σ represents the sum of all training examples from $i=1$ to m .

$$J_{\text{train}}(\theta) = (1/2m) \sum (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Repeat {

$$\theta_j = \theta_j - (\text{learning rate}/m) * \sum (h_{\theta}(x^{(i)}) - y^{(i)})x^{(i)}$$

For every $j = 0 \dots n$

}

Where $x^{(i)}$ Represents the j^{th} feature of the i^{th} training example. So if m is very large (e.g. 5 million training samples), then it takes hours or even days to converge to the global minimum. That's why for large datasets, it is not recommended to use batch gradient descent as it slows down the learning.

Pre Lab Exercise:

- Explain the meaning of linear regression

Linear regression is a method for determining the best linear relationship between two variables X and Y. If variables X and Y are uncorrelated, it is pointless embarking upon linear regression.

- Write three applications of linear regression

- Stock Market Trend Prediction
- Weather Parameter Estimation
- Demand Forecasting in Supply Chain Management

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No: 92301733022

- c. Write three advantages of linear regression
 - Simple and Easy to understand
 - Fast to train and interpret
 - Works well when relationship between variables is approximately linear
- d. Write three limitations of linear regression
 - Fails for non-linear data
 - Sensitive to Outliers
 - Performance drops with multicollinearity and irrelevant features

Methodology:

1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Pre-process the data
5. Visualize the Data
6. Separate the feature and prediction value columns
7. Write the Hypothesis Function
8. Write the Cost Function
9. Write the Gradient Descent optimization algorithm
10. Apply the training over the dataset to minimize the loss
11. Find the best fit line to the given dataset
12. Observe the cost function vs iterations learning curve

Program (Code):

Simple Linear Regression:

Code:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
def estimate_coef(x, y):
# number of
observations/points n =
np.size(x)
```

```
# mean of x and y
vector m_x =
np.mean(x)
m_y = np.mean(y)
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No: 92301733022

calculating cross-deviation and deviation about

```
x SS_xy = np.sum(y*x) - n*m_y*m_x
```

```
SS_xx = np.sum(x*x) - n*m_x*m_x
```

calculating regression

```
coefficients b_1 = SS_xy /
```

```
SS_xx
```

```
b_0 = m_y - b_1*m_x
```

```
return (b_0, b_1)
```

```
def plot_regression_line(x, y, b):
```

```
# plotting the actual points as scatter plot
```

```
plt.scatter(x, y, color = "m", marker = "o", s = 30)
```

```
# predicted response vector
```

```
y_pred = b[0] + b[1]*x
```

```
# plotting the regression line
```

```
plt.plot(x, y_pred, color = "g")
```

```
# putting labels
```

```
plt.xlabel('x')
```

```
plt.ylabel('y')
```

```
plt.show()
```

```
def main():
```

```
# observations / data
```

```
x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
y = np.array([1, 3, 2, 5, 7, 8, 8, 9, 10, 12])
```

```
# estimating
```

```
coefficients b =
```

```
estimate_coef(x, y)
```

```
print("Estimated coefficients:\nb_0 = {}\nb_1 = {}".format(b[0], b[1]))
```

```
# plotting regression line
```

```
plot_regression_line(x, y, b)
```

```
main()
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No: 92301733022

Multiple Linear Regression:

```

from sklearn.model_selection import
train_test_split import matplotlib.pyplot as plt
import numpy as np
from sklearn import datasets, linear_model,
metrics import pandas as pd

data_url = "https://lib.stat.cmu.edu/datasets/boston"
raw_df = pd.read_csv(data_url, sep=r"\s+", skiprows=22, header=None)

X = np.hstack([raw_df.values[::2, :], raw_df.values[1::2,
:2]]) y = raw_df.values[1::2, 2]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4,

random_state=1) reg = linear_model.LinearRegression()
reg.fit(X_train, y_train)

# regression coefficients
print('Coefficients: ', reg.coef_)

# variance score: 1 means perfect prediction
print('Variance score: {}'.format(reg.score(X_test,
y_test)))

# plot for residual

error # setting plot

style
plt.style.use('fivethirtyeight')

# plotting residual errors in training data
plt.scatter(reg.predict(X_train),
            reg.predict(X_train) -
            y_train, color="green",
            s=10, label='Train data')

# plotting residual errors in test data
plt.scatter(reg.predict(X_test),
            reg.predict(X_test) - y_test,
            color="blue", s=10,
            label='Test data')

```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No: 92301733022

```
# plotting line for zero residual error
plt.hlines(y=0, xmin=0, xmax=50, linewidth=2)
```

```
# plotting legend
plt.legend(loc='upper
right')
```

```
# plot title
plt.title("Residual
errors")
```

```
# method call for showing the plot
plt.show()
```

Polynomial Linear

Regression #Polynomial

Linear Regression import pandas

```
as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn
import warnings
```

```
from sklearn.preprocessing import LabelEncoder
from sklearn.impute import KNNImputer
from sklearn.model_selection import
train_test_split from sklearn.preprocessing
import StandardScaler from sklearn.metrics
import f1_score
from sklearn.ensemble import
RandomForestRegressor from sklearn.ensemble
import RandomForestRegressor from
sklearn.model_selection import cross_val_score
```

```
warnings.filterwarnings('ignore')
```

```
df = pd.read_csv('Position_Salaries.csv')
```

```
X = df.iloc[:,1:2].values # Corrected to select 'Level' column as a
2D array y = df.iloc[:,2].values # Corrected to select 'Salary'
column
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No: 92301733022

```

from sklearn.linear_model import
LinearRegression lin_reg=LinearRegression()
lin_reg.fit(X,y)

```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No: 92301733022

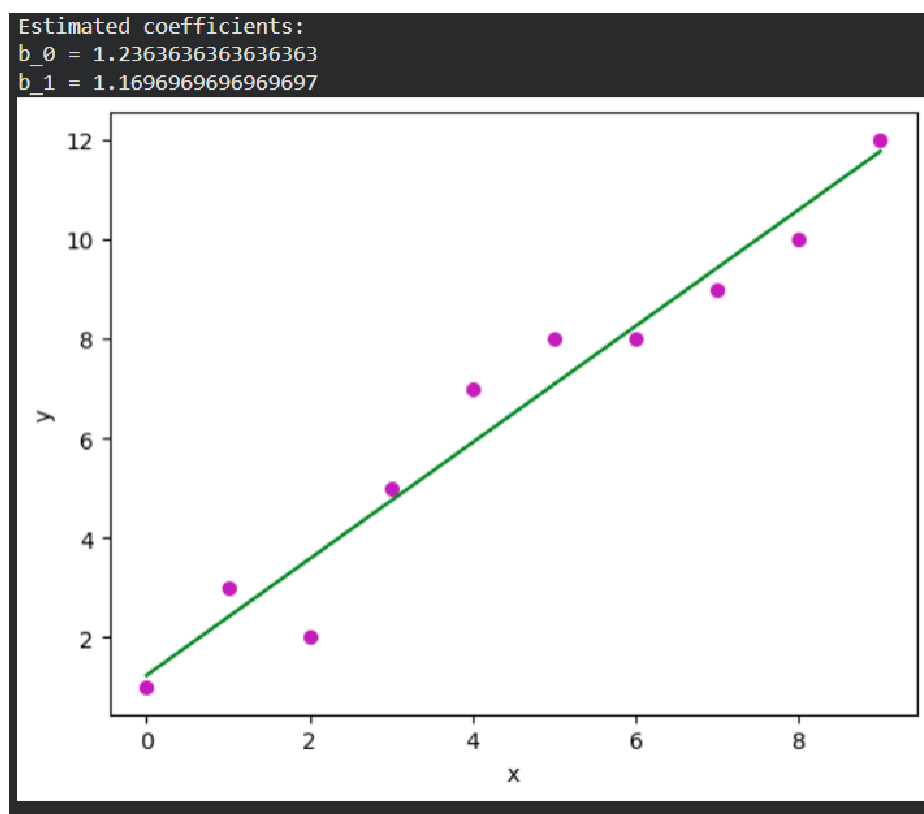
```
from sklearn.preprocessing import PolynomialFeatures
poly_reg2=PolynomialFeatures(degree=2)
X_poly=poly_reg2.fit_transform(X)
lin_reg_2=LinearRegression()
lin_reg_2.fit(X_poly,y)
```

```
poly_reg3=PolynomialFeatures(degree=3)
X_poly3=poly_reg3.fit_transform(X)
lin_reg_3=LinearRegression()
lin_reg_3.fit(X_poly3,y)
```

```
plt.scatter(X,y,color='red')
plt.plot(X,lin_reg.predict(X),color='green')
plt.title('Simple Linear Regression')
plt.xlabel('Position Level')
plt.ylabel('Salary')
plt.show()
```

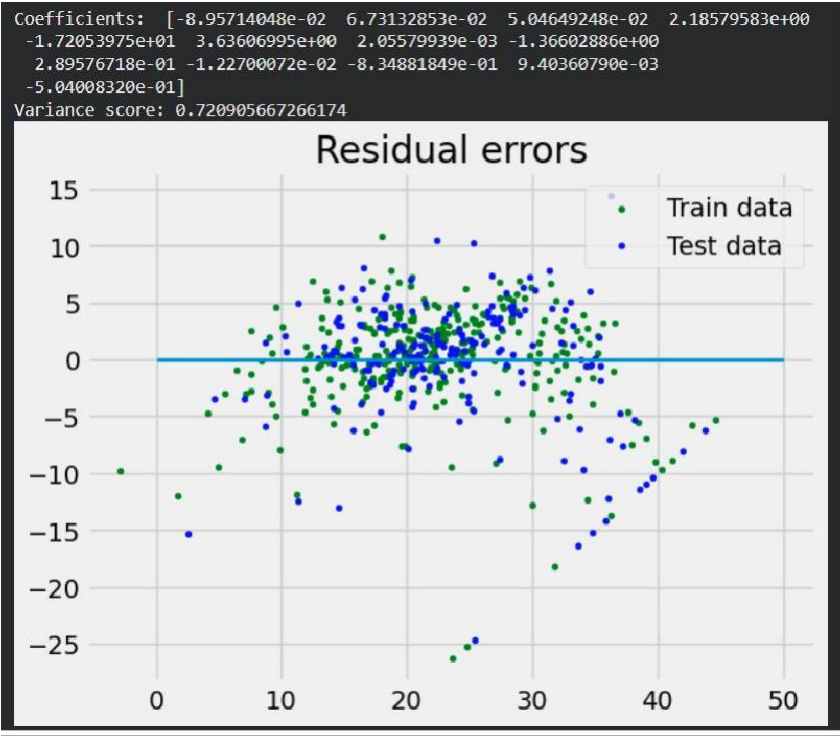
Results:

Simple Linear Regression:

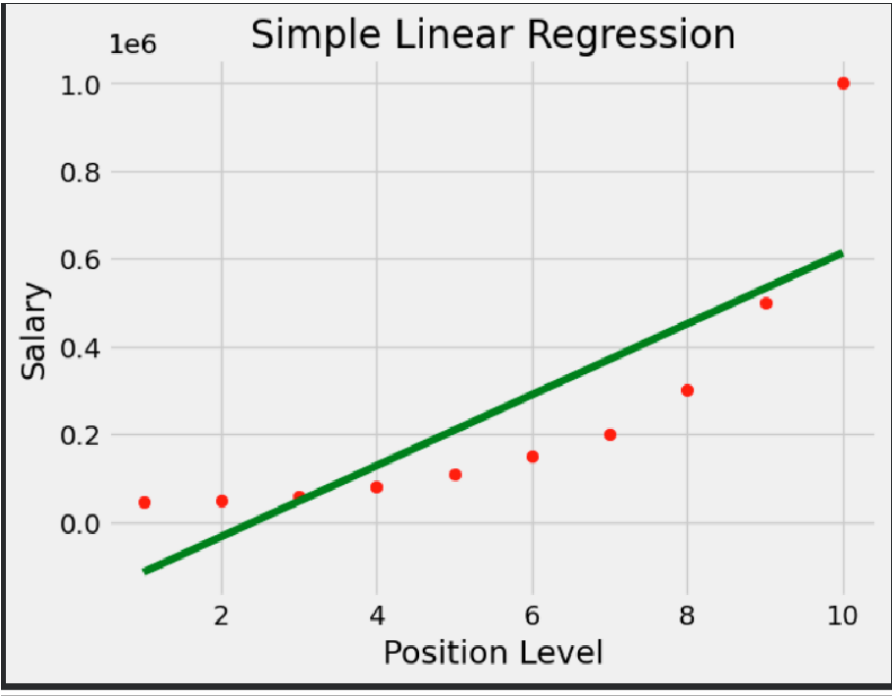


 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No: 92301733022

Multiple Linear Regression:



Polynomial Linear Regression:



 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No: 92301733022

Observation and Result Analysis:

a. Nature of the dataset

- Simple Linear Regression: small, univariate, continuous, labeled dataset.
- Multiple Linear Regression: multivariate, numerical, labeled, real-world dataset.
- Polynomial Linear Regression: small, univariate, numerical, labeled dataset

b. During Training Process

○ Simple Linear Regression:

- Supervised and labeled (each x has a y)
- Numerical and continuous
- Univariate (1 input, 1 output)
- Structured/tabular
- Used to learn a linear relationship

○ Multiple Linear Regression:

- Supervised and labeled
- Numerical and continuous
- Multivariate (many input features, one output)
- Structured/tabular
- Represents a linear relationship in higher dimensions

○ Polynomial Linear Regression:

- Supervised and labeled
- Numerical and continuous
- Univariate (in your case)
- Structured/tabular
- Non-linear in original form, but converted into polynomial features
- Model still trained using linear regression on transformed data

c. After the training Process

○ Simple Linear Regression:

- The dataset is represented by a learned linear model:

$$y = b_0 + b_1x$$

- Data can now be:

- Predicted dataset ($\hat{y}$ values)
- Residual dataset (errors = $y - \hat{y}$)

- Nature: Numerical, continuous, model-fitted dataset used for prediction and evaluation.

○ Multiple Linear Regression:

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No: 92301733022

- The dataset is represented by a multivariate linear equation:

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$
- Dataset is split into:
 - Predicted values
 - Residuals/errors
- Nature: Multivariate numerical, continuous, trained regression output dataset.
- Polynomial Linear Regression:
 - Original data is transformed into polynomial features
 - The final dataset follows a non-linear fitted curve
 - Used to generate:
 - Predicted salaries
 - Error/residual values
 - Nature: Numerical, continuous, non-linearly fitted regression dataset.

Post Lab Exercise:

- a. What are the major assumptions considered in linear regression
 - Linearity
 - Independence of Errors
 - Normality of error
 - No Multicollinearity
 - No or Little Outliers
- b. Why MSE is used instead of MAE for calculating the loss function

Large errors are penalized more heavily since MSE squares the errors. This helps the model concentrate on minimizing large errors.

Differentiable everywhere

Because MSE is differentiable and smooth, gradient descent optimization is simpler. At zero, MAE is not differentiable.

 - Ensures a single optimal solution for linear regression by providing a unique global minimum MSE and a convex cost function.

Convenient in mathematics

Stable convergence and closed-form solutions (Normal Equation) are the results of MSE.
- c. How can the behavior of outliers be understood while dealing with the unseen dataset

Examining residuals, or forecast errors

On unobserved data, large residuals suggest poor generalization or potential outliers.

Making use of visualization methods

Abnormal patterns or extreme values can be found using scatter plots, box plots, and residual plots.

 - You may determine whether unobserved points are significant outliers by looking at statistical

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression	
Experiment No: 02	Date:	Enrollment No: 92301733022

indicators like the Z-score, IQR, and leverage/Cook's distance.

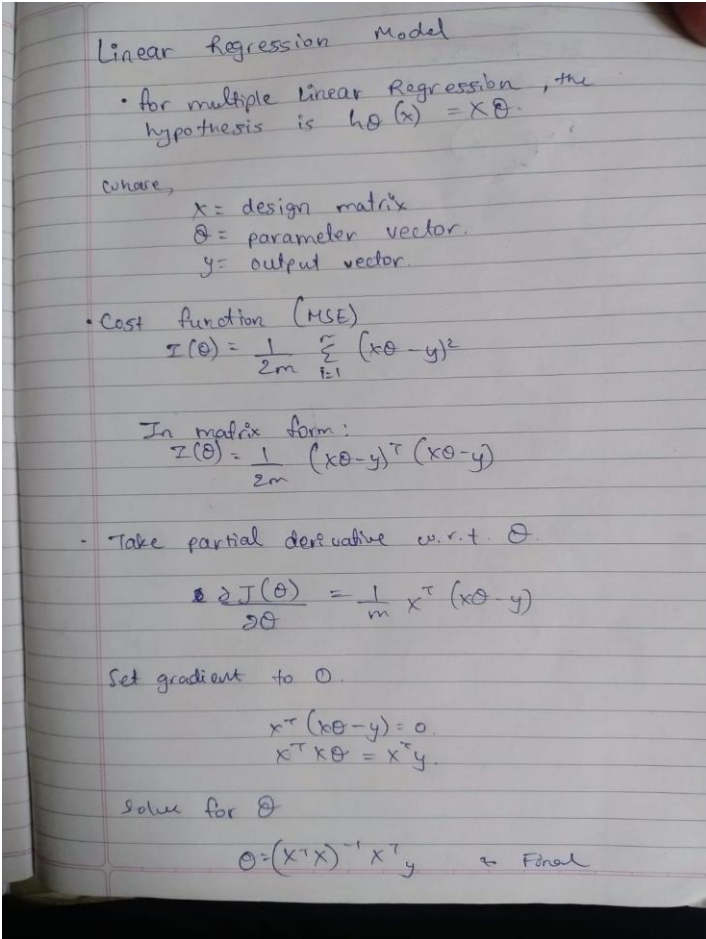
Comparing test and train performance

On unseen data, an abrupt increase in error may indicate outliers or a shift in the distribution.

Testing the robustness of the model

Understanding outlier sensitivity is aided by seeing how forecasts alter when extreme values emerge.

d. Derive the Normal Equation for the Linear Regression.



Linear Regression Model

- For multiple Linear Regression, the hypothesis is $h_{\theta}(x) = X\theta$.

where,

- X = design matrix
- θ = parameter vector
- y = output vector

- Cost Function (MSE)

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (x_i\theta - y_i)^2$$

In matrix form:

$$J(\theta) = \frac{1}{2m} (X\theta - y)^T (X\theta - y)$$

- Take partial derivative w.r.t. θ .

$$\frac{\partial J(\theta)}{\partial \theta} = \frac{1}{m} X^T (X\theta - y)$$

Set gradient to 0.

$$X^T (X\theta - y) = 0$$

$$X^T X \theta = X^T y$$

Solve for θ

$$\theta = (X^T X)^{-1} X^T y \quad \leftarrow \text{Final}$$